



DoS/DDoS - Slowloris Attack and Defense

In contrast to Smurf and TCP SYN flooding attacks which are volumetric-based DoS/DDoS that requires the sending of overwhelming amount of network traffic to flood a target, **Slowloris** attack is an example of **low-rate DoS/DDoS attack** which only needs to send a small amount of maliciously crafted packets to render the target unavailable to legitimate users.

Specifically, **Slowloris** is targeting the design weakness of **HTTP** protocol which requires the HTTP request header to be fully received by the server before it can respond back as shown in Figure 1. Within this duration, the connection will be reserved for the connected client and will remain unavailable to service other clients.

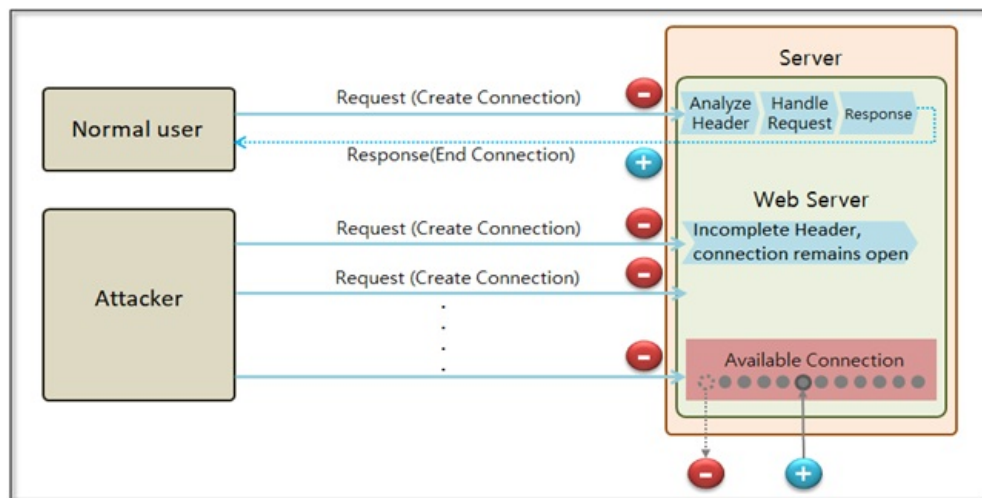


Figure 1

So, what if an attacker were to establish a number of concurrent connections with the server but intentionally sending the incomplete HTTP request header a little bit at a time at a very slow rate like the slow-moving animal slowloris? As a result, all these concurrent connections will remain reserved for the attacker. Sooner or later, all the available connections of the server will be fully taken up and unavailable to service new legitimate user which is the result of **Slowloris attack**.

To let you have a sense of how easy to conduct Slowloris attack and appreciate the importance of configuring defense against it, you will experiment it in the lab using **slowhttptest** tool available in Kali Linux. An example is shown in Figure 2.

```
kali@kali: ~/slowloris
File Actions Edit View Help
Thu Jun 6 21:31:53 2024:
slowhttptest version 1.9.0
- https://github.com/shekyaan/slowhttptest -
test type: SLOW HEADERS
number of connections: 1500
URL: http://192.0.2.2/
verb: GET
cookie:
Content-Length header value: 4096
follow up data max size: 68
interval between follow up data: 10 seconds
connections per seconds: 50
probe connection timeout: 5 seconds
test duration: 240 seconds
using proxy: no proxy

Thu Jun 6 21:31:53 2024:
slow HTTP test status on 240th second:

initializing: 0
pending: 0
connected: 1490
error: 0
closed: 10
service available: NO
Thu Jun 6 21:31:54 2024:
Test ended on 241th second
Exit status: Hit test time limit

(kali@kali)-[~/slowloris]
$
```

Figure 2

Notice that a single attacker's PC is sufficient to bring down a server rendering it unavailable to legitimate users as shown in Figure 3.

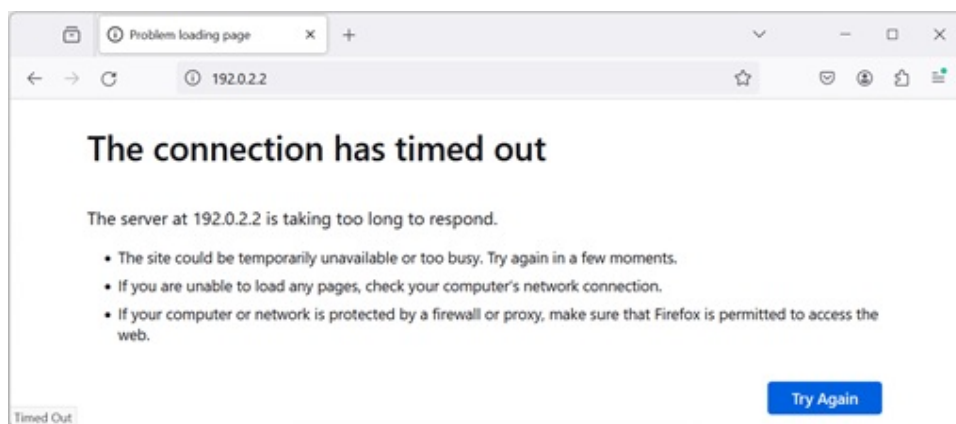


Figure 3

Defending against Slowloris Attack

Unfortunately, ACL is also unable to defend against Slowloris attack. In fact, ACL is intentionally applied to permit users (which will include potential attackers) from the Internet to visit and hence possibly attack the Web server.

Similarly, we need capability to **limit connections** provided by many stateful firewalls including Cisco Firepower/ASA firewalls, which can be configured using **MPF**. For example, consider the network topology where the web server A is hosted at IP address 10.1.1.5 as shown in Figure 4:

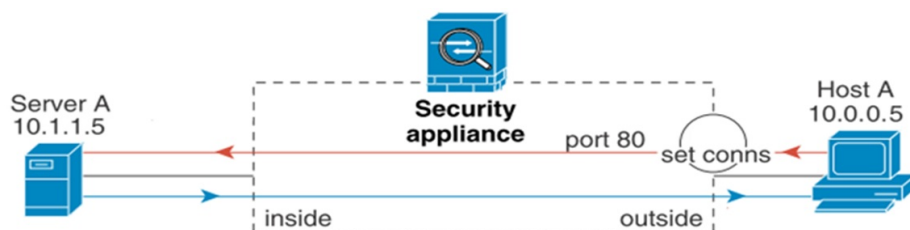


Figure 4

A possible MPF configuration may be written as follows:

```
ciscoasa(config)# access-list accesslist_name extended permit ...
ciscoasa(config)# class-map class_map_name
ciscoasa(config-cmap)# match access-list accesslist_name

ciscoasa(config)# policy-map policy_map_name
ciscoasa(config-pmap)# class class_map_name
ciscoasa(config-pmap-c)# set connection ?
```

For your practice and understanding, go through the different parameters of **set connection** as displayed by **?** and try

them until you are able to defend against Slowloris attack. Remember the need to also configure **service-policy** to apply the policy-map onto the required interface of the firewall.

Explore, learn and enjoy!